코드 구현 세부사항 분석

개요

이 문서는 파일코인 proof 시스템의 핵심 알고리즘들을 실제 코드 구현 관점에서 상세히 분석합니다. 의사코드와 실제 구현을 비교하고, 주요 함수들의 동작 과정을 단계별로 설명하여 개발자들이 구현 세부사항을 깊이 있게 이해할 수 있도록 돕습니다.

1. 레이블 생성 알고리즘 (create_label)

1.1 알고리즘 개요

레이블 생성은 Stacked DRG PoRep의 핵심 구성 요소로, 각 노드에 대해 고유한 레이블을 생성하여 데이터 인코딩에 사용됩니다.

1.2 의사코드

```
function create_label(graph, replica_id, layer_labels, layer_index, node):

hasher = SHA256()

buffer = [0; 32]

// 레이어 인덱스와 노드 인덱스를 버퍼에 저장

buffer[0:4] = layer_index.to_be_bytes()

buffer[4:12] = node.to_be_bytes()

// replica_id와 버퍼를 해시에 입력

hasher.input(replica_id)
```

```
hasher.input(buffer)

if node > 0:

// 부모 노드들의 데이터를 해시에 포함

hash = graph.copy_parents_data(node, layer_labels, hasher)

else:

hash = hasher.finish()

// 생성된 해시를 레이블로 저장

layer_labels[node * NODE_SIZE:(node + 1) * NODE_SIZE] = hash

// Fr 필드 원소로 만들기 위해 마지막 2비트 제거

layer_labels[(node + 1) * NODE_SIZE - 1] &= 0b0011_1111
```

1.3 실제 구현 분석

```
pub fn create_label<H: Hasher, T: AsRef<[u8]>>(

graph: &StackedBucketGraph<H>,

cache: Option<&mut ParentCache>,

replica_id: T,

layer_labels: &mut [u8],

layer_index: usize,

node: usize,

) -> Result<()> {

let mut hasher = Sha256::new();

let mut buffer = [0u8; 32];
```

```
// 레이어 인덱스와 노드 인덱스를 빅엔디안으로 저장
buffer[..4].copy_from_slice(&(layer_index  as u32).to_be_bytes());
buffer[4..12].copy_from_slice(&(node  as u64).to_be_bytes());
hasher.input(&[replica_id.as_ref(), &buffer[..]][..]);


// 노드 0이 아닌 경우 부모 데이터 해싱
let  hash = if  node > 0 {
// 이전 노드(항상 부모)를 프리페치
let  prev = &layer_labels[(node - 1) * NODE_SIZE..node * NODE_SIZE];
prefetch!(prev.as_ptr() as *const i8);


graph.copy_parents_data(node  as u32, &*layer_labels, hasher, cache)?
} else {
hasher.finish()
};


// 새로 생성된 키 저장
let  start = data_at_node_offset(node);
let  end = start + NODE_SIZE;
layer_labels[start..end].copy_from_slice(&hash[..]);


// Fr 필드 원소로 만들기 위해 마지막 2비트 제거
```

```
layer_labels[end - 1] &= 0b0011_1111;



Ok(())

}
```

1.4 확장 레이어 레이블 생성 (create_label_exp)

확장 레이어에서는 이전 레이어의 데이터도 부모로 사용합니다:

```
pub fn create_label_exp<H: Hasher, T: AsRef<[u8]>>(

graph: &StackedBucketGraph<H>,

cache: Option<&mut ParentCache>,

replica_id: T,

exp_parents_data: &[u8], // 이전 레이어 데이터

layer_labels: &mut [u8],

layer_index: usize,

node: usize,

) -> Result<()> {

// 기본 구조는 동일하지만 확장 부모 데이터 사용

graph.copy_parents_data_exp(node as u32, &*layer_labels, exp_parents_data, hash

}
```

2. 데이터 인코딩/디코딩 함수

2.1 인코딩 알고리즘

```
pub fn encode<T: Domain>(key: T, value: T) -> T {
let value: Fr = value.into();
let mut result: Fr = key.into();

encode_fr(&mut result, value);
result.into()
}

pub fn encode_fr(key: &mut Fr, value: Fr) {
*key += value; // 단순한 덧셈 연산
}
```

2.2 디코딩 알고리즘

```
pub fn decode<T: Domain>(key: T, value: T) -> T {
let mut result: Fr = value.into();
let key: Fr = key.into();

result -= key; // 인코딩의 역연산
result.into()
}
```

2.3 인코딩/디코딩 과정 분석

1. **데이터 변환**: 입력 데이터를 유한체(Fr) 원소로 변환
2. **연산 수행**:
   - 인코딩: `encoded = key + value`
   - 디코딩: `decoded = encoded - key`
3. **결과 변환**: 유한체 원소를 원래 도메인으로 변환

이 방식은 단순하지만 효과적인 대칭 암호화 방식으로, 키를 알면 쉽게 복호화할 수 있습니다.

3. 머클 증명 생성 및 검증

3.1 머클 증명 구조

```
pub struct MerkleProof<H, BaseArity, SubTreeArity, TopTreeArity> {
data: ProofData<H, BaseArity, SubTreeArity, TopTreeArity>,
}

enum ProofData<H, BaseArity, SubTreeArity, TopTreeArity> {
Single(SingleProof<H, BaseArity>),
Sub(SubProof<H, BaseArity, SubTreeArity>),
Top(TopProof<H, BaseArity, SubTreeArity, TopTreeArity>),
}
```

3.2 증명 생성 과정

```
impl<H: Hasher, Arity: PoseidonArity> InclusionPath<H, Arity> {
pub fn root(&self, leaf: H::Domain) -> H::Domain {
let mut a = H::Function::default();
(0..self.path.len()).fold(leaf, |h, height| {
a.reset();

let index = self.path[height].index;
let mut nodes = self.path[height].hashes.clone();
nodes.insert(index, h);

a.multi_node(&nodes, height)
})
}
}
```

3.3 증명 검증 알고리즘

```
impl<H: Hasher, Arity: PoseidonArity> SingleProof<H, Arity> {
fn verify(&self) -> bool {
let calculated_root = self.path.root(self.leaf);
self.root == calculated_root
}
```

```
}
```

3.4 검증 과정 단계별 분석

1. **리프에서 시작**: 증명하려는 데이터(리프)부터 시작
2. **경로 따라 올라가기**: 각 레벨에서 형제 노드들과 함께 해시 계산
3. **루트 계산**: 최종적으로 계산된 루트와 저장된 루트 비교
4. **검증 완료**: 두 루트가 일치하면 증명 유효

4. Feistel 암호를 이용한 부모 노드 생성

4.1 Feistel 암호 구조

```
pub const FEISTEL_ROUNDS: usize = 3;


fn encode(index: Index, keys: &[Index], precomputed: FeistelPrecomputed) -> Ind
let (mut left, mut right, right_mask, half_bits) = common_setup(index, precomp


for key in keys.iter().take(FEISTEL_ROUNDS) {
let (l, r) = (right, left ^ feistel(right, *key, right_mask));
left = l;
right = r;
}


(left << half_bits) | right
```

```
}
```

4.2 Feistel 라운드 함수

```
fn feistel(right: Index, key: Index, right_mask: Index) -> Index {
let mut data: [u8; FEISTEL_BYTES] = [0; FEISTEL_BYTES];
// right와 key를 바이트 배열로 변환
// ... (바이트 변환 코드)
let raw = blake2b(&data);
let hash = raw.as_bytes();
// 해시 결과를 Index로 변환하고 마스크 적용
let result = /* 해시를 Index로 변환 */;
result & right_mask
}
```

4.3 부모 노드 생성 과정

```
impl<H, G> StackedGraph<H, G> {
fn correspondent(&self, node: usize, i: usize) -> u32 {
let a = (node * self.expansion_degree) as feistel::Index + i as feistel::Index

let transformed = feistel::permute(
self.size() as feistel::Index * self.expansion_degree as feistel::Index,
```

```
a,
&self.feistel_keys,
self.feistel_precomputed,
);

// 변환된 인덱스를 노드 인덱스로 매핑
transformed as u32 / self.expansion_degree as u32
}
}
```

5. DRG 부모 생성 알고리즘

5.1 버킷 샘플링 알고리즘

```
impl<H: Hasher> Graph<H> for BucketGraph<H> {
fn parents(&self, node: usize, parents: &mut [u32]) -> Result<()> {
match node {
0 | 1 => {
// 첫 번째와 두 번째 노드는 특별 처리
for parent in parents.iter_mut().take(self.degree()) {
*parent = 0;
}
}
```

```
_ => {
let node = node as u32;
let mut rng = ChaCha8Rng::from_seed(/* 시드 생성 */);
let m_prime = self.degree() - 1;
let metagraph_node = node as u64 * m_prime as u64;
let n_buckets = (metagraph_node as f64).log2().ceil() as u64;


for parent in parents.iter_mut().take(m_prime) {
let bucket_index = (rng.gen::<u64>() % n_buckets) + 1;
let largest_distance = min(metagraph_node, 1 << bucket_index);
let smallest_distance = max(2, largest_distance >> 1);
let distance = smallest_distance +
(rng.gen::<u64>() % (largest_distance - smallest_distance + 1));
let metagraph_parent = metagraph_node - distance;
let mapped_parent = (metagraph_parent / m_prime as u64) as u32;
*parent = if mapped_parent == node { node - 1 } else { mapped_parent };
}
// 직전 노드를 첫 번째 부모로 설정
parents[0] = node - 1;
}
}
Ok(())
}
```

```
}
```

5.2 버킷 샘플링 과정 분석

1. **메타그래프 매핑**: DRG 노드를 메타그래프 노드로 매핑
2. **버킷 수 계산**: `log2(metagraph_node)` 로 버킷 수 결정
3. **버킷 선택**: 랜덤하게 버킷 선택
4. **거리 범위 계산**: 선택된 버킷 내에서 거리 범위 결정
5. **부모 선택**: 거리 범위 내에서 랜덤하게 부모 선택
6. **매핑**: 메타그래프 부모를 DRG 부모로 매핑

6. 성능 최적화 기법

6.1 메모리 프리페칭

```
#[inline]
fn prefetch(parents: &[u32], data: &[u8]) {
for parent in parents {
let start = *parent as usize * NODE_SIZE;
let end = start + NODE_SIZE;
prefetch!(data[start..end].as_ptr() as *const i8);
}
}
```

6.2 부모 캐시 시스템

```
impl<H, G> StackedGraph<H, G> {
pub fn parent_cache(&self) -> Result<ParentCache> {
let default_cache_size = SETTINGS.sdr_parents_cache_size;
let cache_entries = self.size() as u32;
let cache_size = cache_entries.min(default_cache_size);
ParentCache::new(cache_size, cache_entries, self)
}
}
```

6.3 병렬 처리 최적화

```
pub fn from_par_iter<I>(par_iter: I) -> Result<Self>
where
I: IntoParallelIterator<Item = H::Domain>,
I::Iter: IndexedParallelIterator,
{
let tree = MerkleTree::from_par_iter(par_iter)?;
Ok(tree.into())
}
```

7. 해시 함수 최적화

7.1 SHA256 최적화

```
fn copy_parents_data_inner(
&self,
cache_parents: &[u32],
base_data: &[u8],
mut hasher: Sha256,
) -> [u8; 32] {
// 부모 데이터 프리페치
prefetch(&cache_parents[..BASE_DEGREE], base_data);

// 부모 데이터 읽기
let parents = [/* 부모 데이터 배열 */];

// 다중 라운드 해싱
hasher.input(&parents); // 라운드 1
hasher.input(&parents); // 라운드 2
// ... 추가 라운드들
hasher.finish_with(parents[0])
}
```

7.2 해시 라운드 최적화

- **라운드 1-6**: 6개 부모 데이터를 반복적으로 해싱

- **라운드 7**: 첫 번째 부모로 마무리
- **메모리 접근 최적화**: 프리페칭으로 캐시 미스 최소화

8. 오류 처리 및 검증

8.1 입력 검증

```
pub fn new(/* 매개변수들 */) -> Result<Self> {
ensure!(nodes <= u32::MAX as usize, "too many nodes");
ensure!(expansion_degree == EXP_DEGREE, "invalid expansion degree");
// 추가 검증들...
}
```

8.2 증명 검증

```
impl MerkleProofTrait for MerkleProof<H, Arity, SubTreeArity, TopTreeArity> {
fn validate(&self, node: usize) -> bool {
if !self.verify() {
return false;
}
node == self.path_index()
}
}
```

9. 구현 특징 및 최적화 포인트

9.1 메모리 효율성

- 레이어별 데이터를 디스크에 저장하여 메모리 사용량 최소화
- 부모 캐시를 통한 반복 계산 방지

9.2 계산 효율성

- Feistel 암호의 사전 계산된 매개변수 사용
- 병렬 처리를 통한 머클 트리 구축 가속화

9.3 보안 고려사항

- Fr 필드 원소로의 변환을 통한 안전한 연산
- 암호학적으로 안전한 해시 함수 사용

결론

파일코인의 proof 시스템은 복잡한 암호학적 알고리즘들을 효율적으로 구현하기 위해 다양한 최적화 기법을 사용합니다. 레이블 생성, 데이터 인코딩, 머클 증명, 부모 노드 생성 등의 핵심 알고리즘들은 모두 성능과 보안을 동시에 고려하여 설계되었으며, 실제 구현에서는 메모리 효율성과 병렬 처리를 통한 성능 향상에 중점을 두고 있습니다.